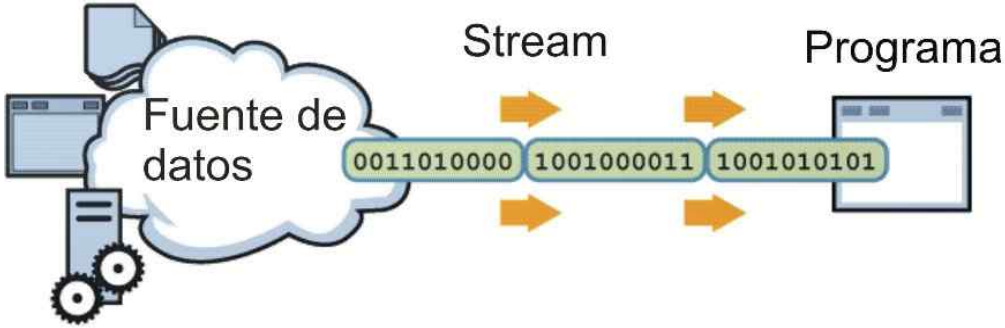
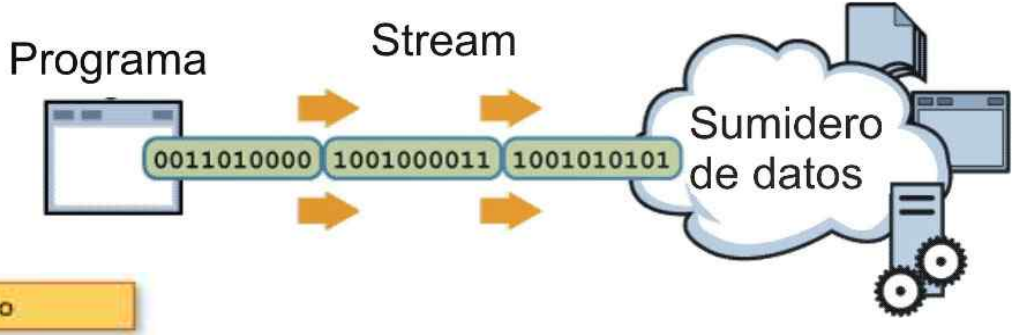
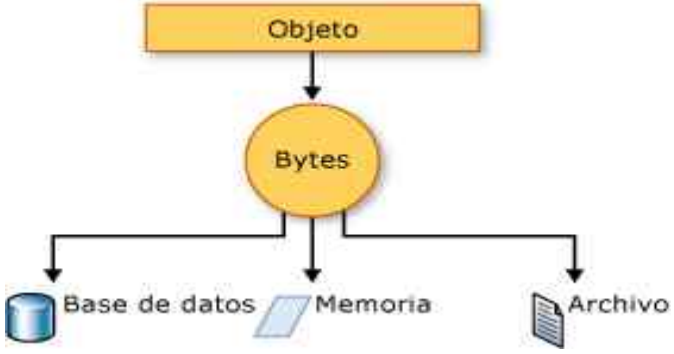
Clase 9 – Archivos, Serialización, XML

- Entrada y Salida de Datos
- Streams orientados a bytes (de bajo nivel y filtrados)
- Streams orientados a caracteres
- Serialización
- Almacenamiento en archivos
- La interfaz serializable
- Herencia y serialización
- XML









Entrada / Salida de Datos en Java



La E/S en Java está basada en la noción de streams (flujos).

> Un flujo es una abstracción de cualquier secuencia de datos yendo desde una fuente (o productor) hacia un destino (o consumidor). Se puede interpretar como una serie de datos circulando a través de un determinado canal que une a la fuente y al destino.

Los bytes se pueden procesar de uno en uno o en bloques (utilizando algún buffer)

En algunos casos se conoce el número de bytes a leer (como en el caso de un archivo que tiene un tamaño) y en otros casos no se conoce el número de bytes a procesar (por ejemplo en una conexión de red)

Entrada / Salida de Datos en Java

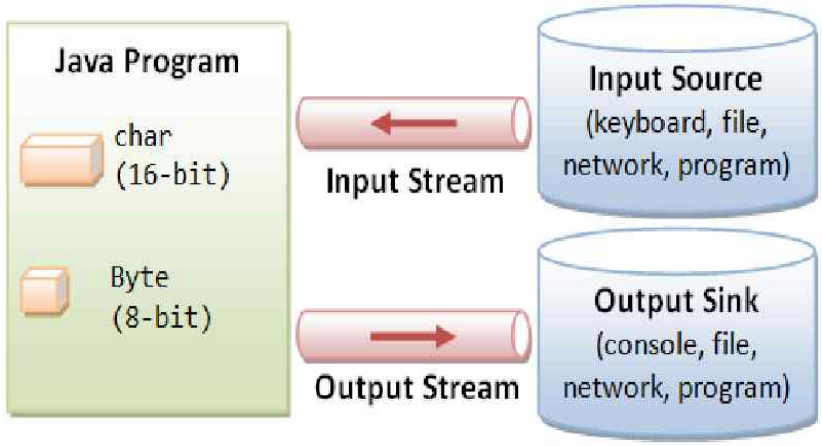
Las clases base del paquete java.io son:

Para flujos orientados a bytes:

- java.io.InputStream
- java.io.OutputStream

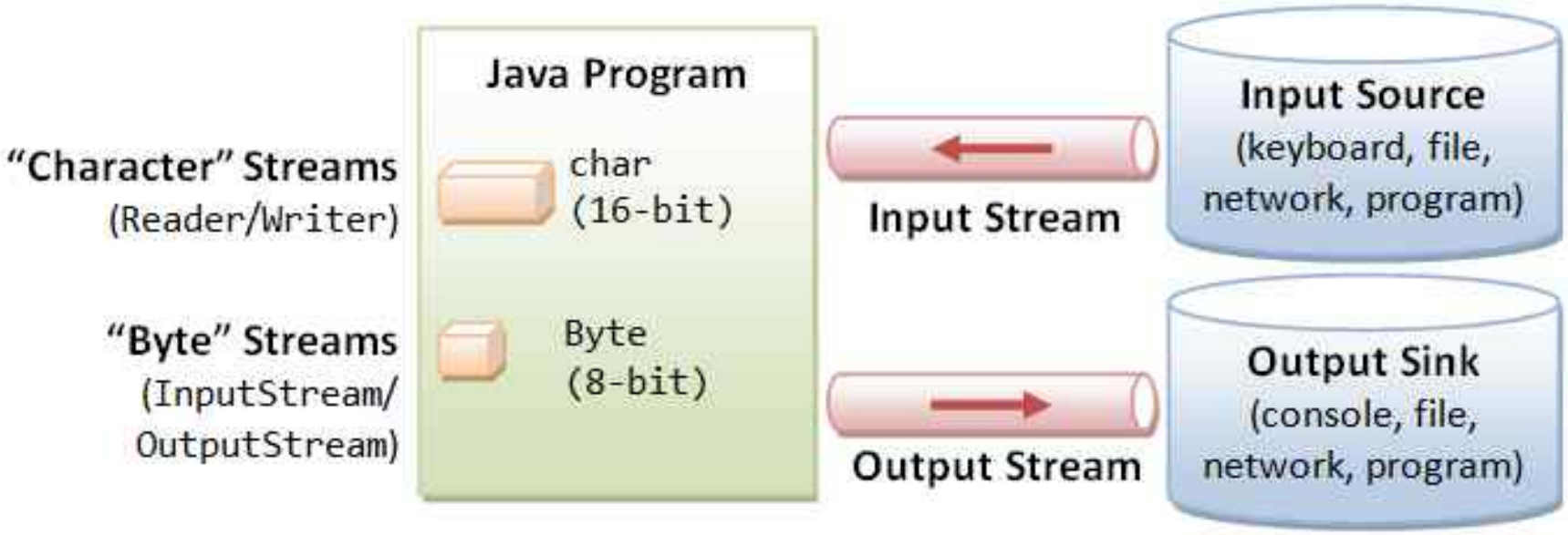
Para flujos orientados a caracteres:

- java.io.Reader
- java.io.Writer



Las cuatro clases anteriores están declaradas como abstractas.

Entrada / Salida de Datos en Java



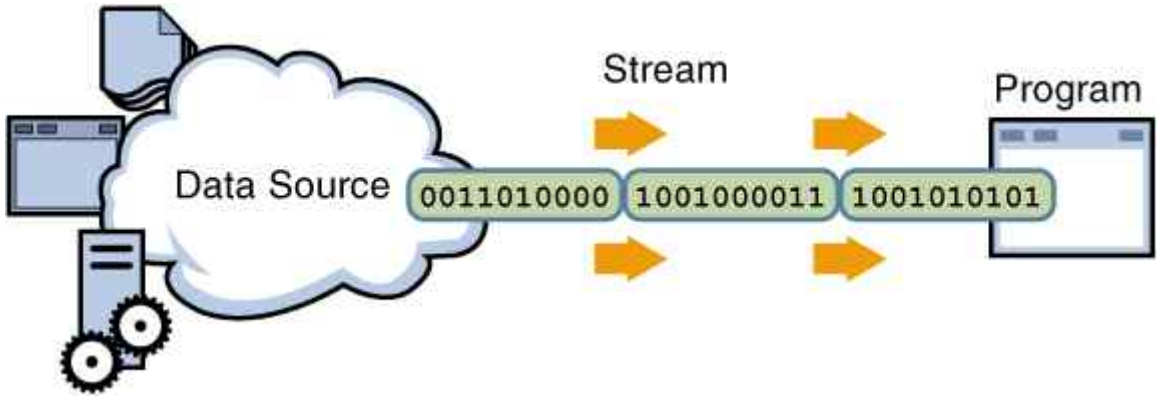
Internal Data Formats:

- Text (char): UCS-2
- int, float, double, etc.

External Data Formats:

- Text in various encodings (US-ASCII, ISO-8859-1, UCS-2, UTF-8, UTF-16, UTF-16BE, UTF16-LE, etc.)
- Binary (raw bytes)

Entrada orientada a bytes

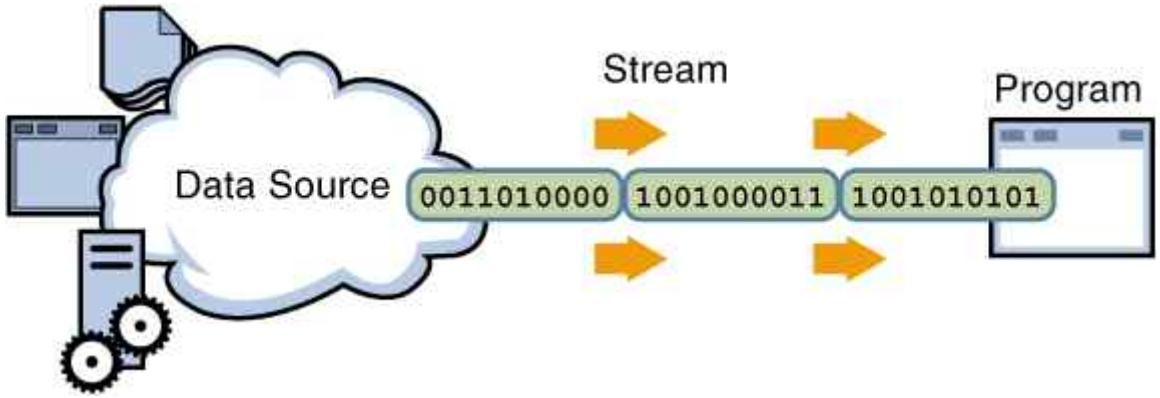


- Los flujos de lectura extienden a la clase **`InputStream`**
- Esta clase define el método abstracto:

```
public abstract int read() throws IOException
```

  - Lee el siguiente byte y lo devuelve como un int en el rango 0 a 255.
  - Si no hay más datos (porque se ha llegado al final) devuelve un -1 (este hecho se utiliza como condición de final).

Entrada orientada a bytes



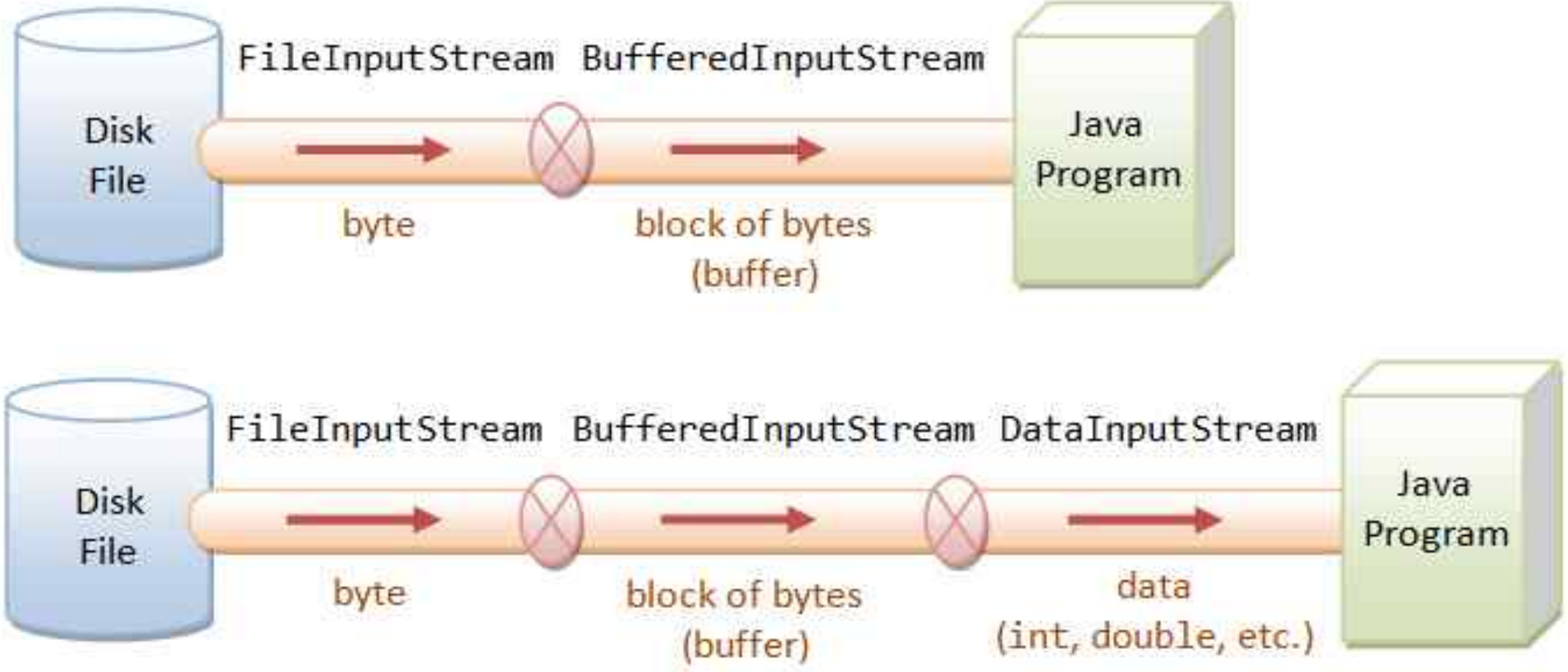
- Este método (**`read()`**) bloquea la ejecución del programa esperando a que se de alguna de las circunstancias siguientes: el siguiente dato esta disponible para ser leído, se detecta el final, se lanza una excepción.
- Cada subclase debe proporcionar una implementación de este método.
- El paquete java.io proporciona unos cuantos flujos de entrada y hay que saber cual es el apropiado para realizar de forma efectiva la lectura.

Entrada orientada a bytes

Tanto para la entrada y la salida orientadas a bytes como las orientadas a caracteres veremos que en la biblioteca java.io se ofrecen dos subconjuntos de clases:

- **De bajo nivel** ofrece funcionalidad básica para la lectura y la escritura de bytes o de caracteres.
- **Filtrada** ofrece alguna funcionalidad añadida para la escritura y lectura de bytes o caracteres.

Entrada orientada a bytes



Entrada de bajo nivel orientada a bytes

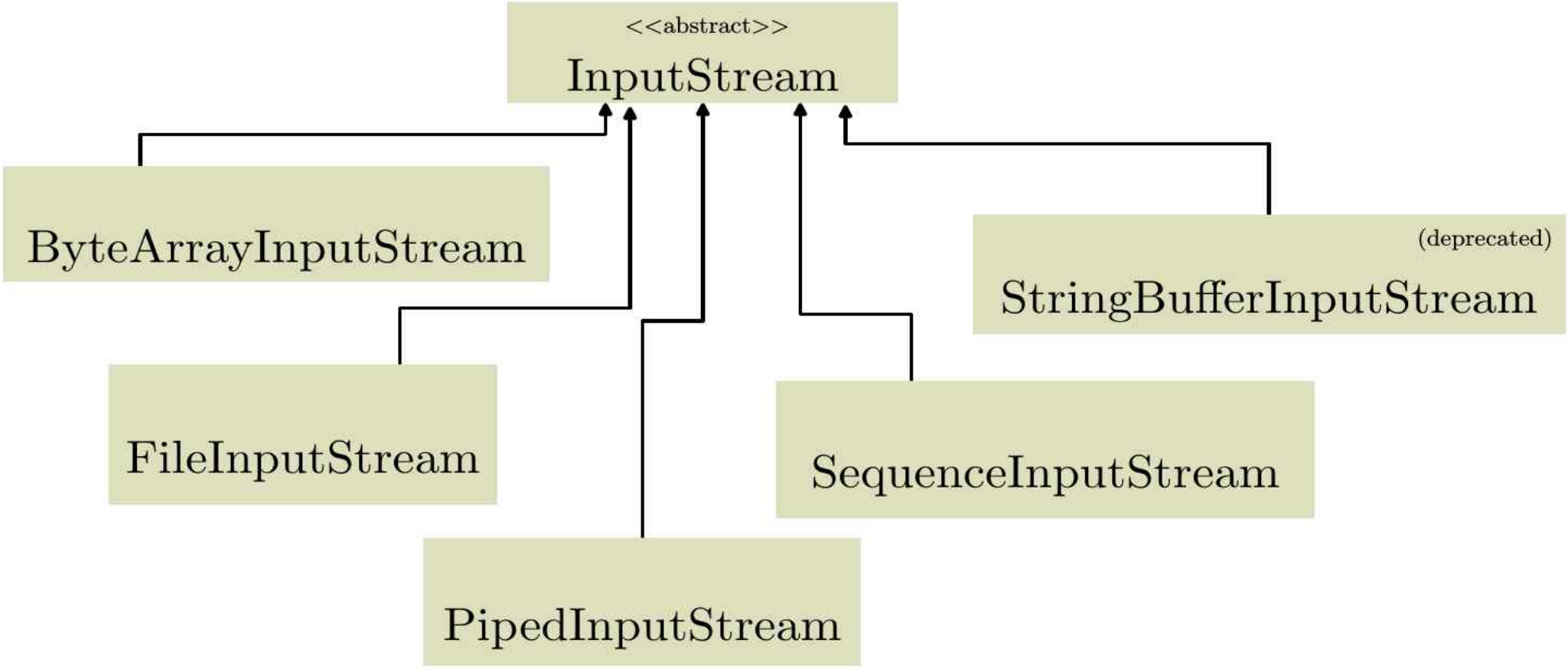
Entrada de bajo nivel orientada a bytes

La siguiente tabla muestra 5 clases que representan flujos de entrada de bajo nivel y que representan implementaciones a **`InputStream`** para diferentes propósitos.

Flujo de entrada	Descripción
ByteArrayInputStream	permite leer bytes de un array que esté en memoria.
FileInputStream	permite leer bytes de un fichero del sistema local de ficheros.
PipedInputStream	permite leer bytes desde un *pipe* creado por un hilo.
StringBufferInputStream	permite leer lee bytes desde un String.
SequenceInputStream	permite leer bytes de datos desde dos o más flujos de bajo nivel, cambiando a otro flujo al finalizar la lectura de cada uno de ellos.

Entrada de bajo nivel orientada a bytes

Entrada de bajo nivel orientada a bytes



Ejemplo

```
public class Demo1
{
    public static void main( String[] args)
    {
        int dato;
        try
        {
            InputStream entrada = new FileInputStream(args[0]);
            while(( dato = entrada.read() )!= -1)
                System.out.write(dato);
            System.out.println();
            entrada.close();
            catch(IOException e)
            {
                . . .
```

Esta no es la forma más eficiente de mostrar el contenido de un archivo, pero sirve para ilustrar la utilización de los flujos de entrada de bajo nivel.

Entrada filtrada orientada a bytes

- A pesar de que los flujos de bajo nivel mostrados permiten leer bytes su flexibilidad es limitada.
- Qué ocurre si deseamos leer una palabra? ¿o un entero? ¿o un carácter?
- Los flujos filtrados (Filter Stream) añaden funcionalidad a un flujo existente, procesando los datos de alguna forma (por ejemplo almacenando los datos en un buffer para mejorar la eficiencia) u ofreciendo métodos adicionales que permitan acceder a los datos de otros modos (por ejemplo leyendo una línea de texto en lugar de byte a byte).

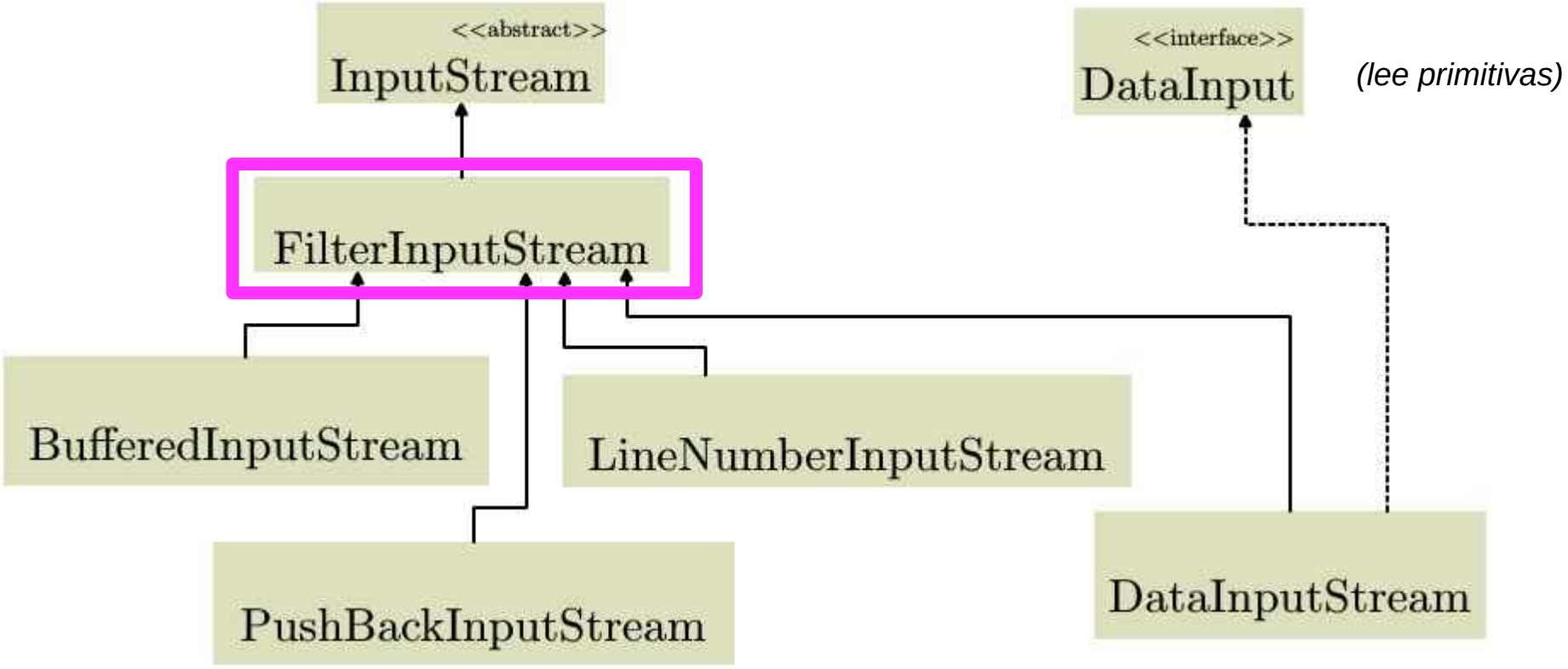
Entrada filtrada orientada a bytes

Las clases que se definen en el paquete java.io para entrada mediante flujos filtrados son:

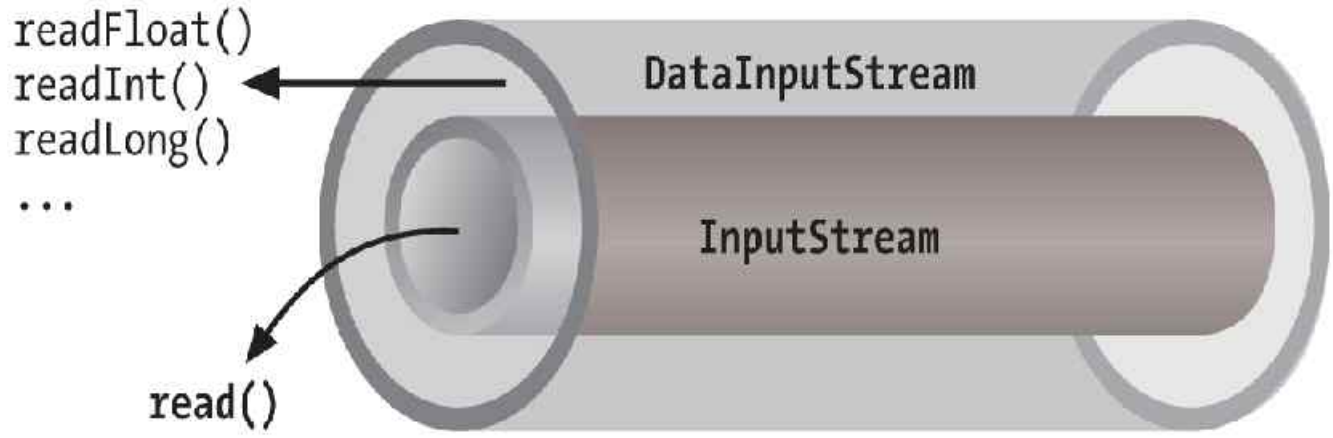
Flujo filtrado	Descripción
BufferedInputStream	almacena el acceso a los datos para mejorar la eficiencia.
DataInputStream	permite leer tipos de datos primitivos, tales como int, float, double e incluso líneas de texto.
LineNumberInputStream	mantiene un contador sobre el número de línea que se está leyendo, basado en una interpretación de los caracteres de final de línea.
PushBackInputStream	añade a otro flujo de entrada la capacidad de volver a colocar un byte leído (o array de bytes) otra vez en el flujo. La siguiente operación de lectura volverá a leer el último byte.

Entrada filtrada orientada a bytes

Las clases que se definen en el paquete java.io para entrada mediante flujos filtrados son:



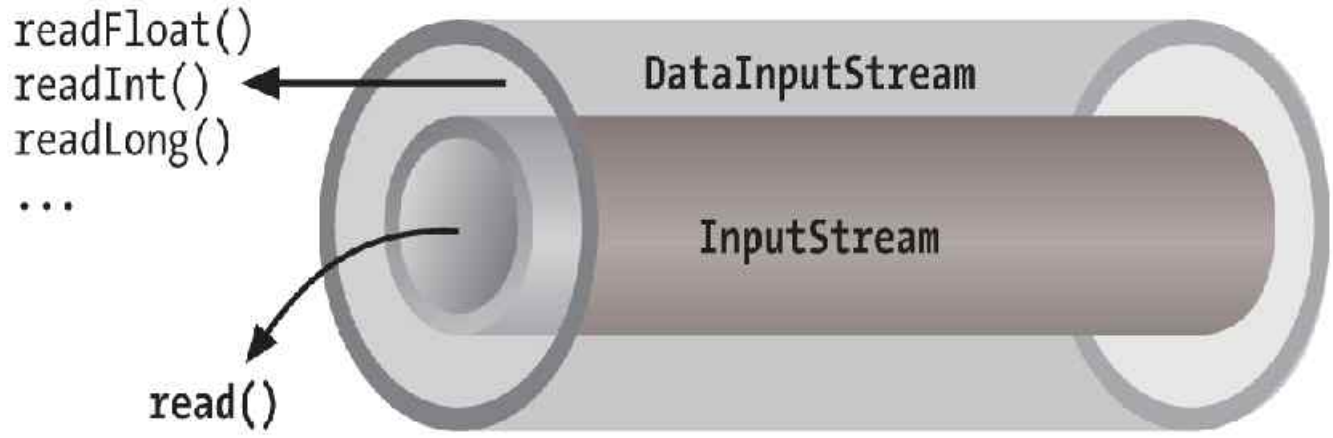
Entrada filtrada orientada a bytes: FilterInputStream



El constructor de esta clase admite otro flujo de entrada, que es utilizado como fuente de datos (la lectura se realiza a través de éste), posiblemente transformando los datos conforme son leídos o proporcionando una funcionalidad adicional que no poseía el flujo original. El constructor es:

```
protected FilterInputStream(InputStream in)
```

Entrada filtrada orientada a bytes: FilterInputStream



La clase FilterInputStream simplemente oculta todos los métodos de InputStream con versiones que lo que realizan es pasar las peticiones al flujo de entrada con el que se creó.

Las subclases de FilterInputStream pueden ocultar algunos de estos métodos y pueden proporcionar métodos o atributos adicionales.

Esta solución, se conoce como patrón Decorator y pertenece al grupo de patrones estructurales.

Entrada filtrada orientada a bytes: FilterInputStream

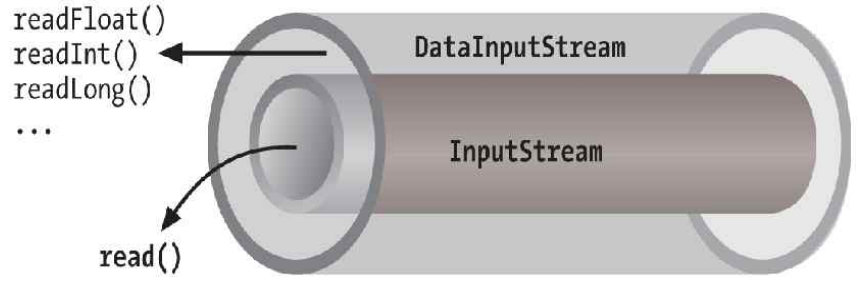
`protected FilterInputStream(InputStream in)`

```
// A partir del array de bytes se obtiene un ByteArrayInputStream
// a partir de este un BufferedInputStream y a partir de este un
// DataInputStream
DataInputStream in = new DataInputStream(
    new BufferedInputStream(new ByteArrayInputStream(bytes)));


try{
  while (true){
    System.out.print(in.readChar());
  }
```

Flujo filtrado
BufferedInputStream
DataInputStream
LineNumberInputStream
PushBackInputStream

Flujo de entrada
ByteArrayInputStream
FileInputStream
PipedInputStream
StringBufferInputStream
SequenceInputStream



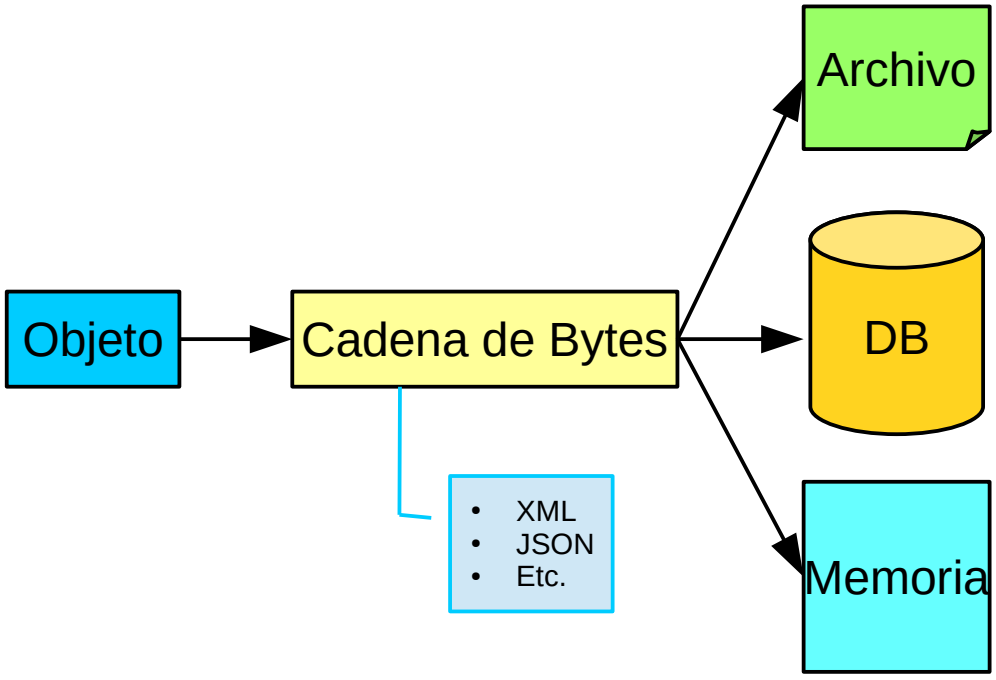
Leer los siguientes temas

- Salida orientada a bytes
  - Salida de bajo nivel orientada a bytes
  - Utilización de un flujo de salida de bajo nivel orientado a bytes

  - Salida filtrada orientada a bytes
  - Utilización de un flujo de salida filtrado orientado a bytes

- Entrada y Salida orientada a caracteres

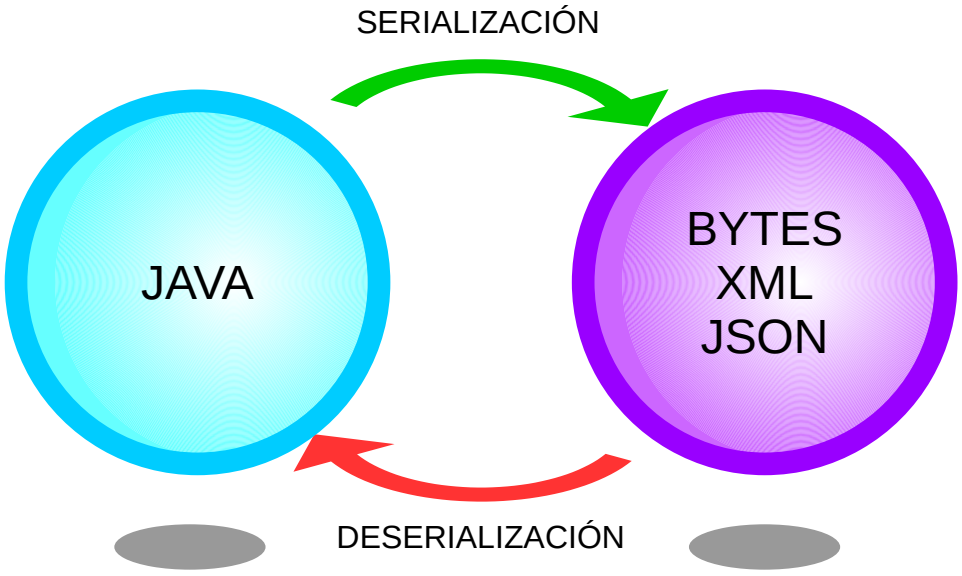
Serialización



La serialización está orientada a bytes *por lo tanto se utilizan clases que estén en la jerarquía de InputStream u OutputStream.*

En ciencias de la computación, la serialización (o marshalling en inglés) consiste en un **proceso de codificación** de un objeto en un medio de almacenamiento (como puede ser un archivo, o un buffer de memoria) o con el fin de transmitirlo a través de una conexión en red como una **serie de bytes** o en un **formato** humanamente **más legible** como **XML** o JSON, entre otros.

Serialización



La serie de bytes o el formato pueden ser usados para crear un nuevo objeto que es idéntico en todo al original, incluido su **estado interno** (por tanto, el nuevo objeto es un clon del original). La serialización es un mecanismo ampliamente usado para transportar objetos a través de una red, para hacer persistente un objeto en un archivo o base de datos, o para distribuir objetos idénticos a varias aplicaciones o localizaciones.

Serialización

La serialización es una característica añadida al lenguaje Java para dar soporte a:

La invocación remota de objetos

permite a los objetos que viven en otras computadoras comportarse como si vivieran en la propia máquina. La serialización es necesaria para transportar los argumentos y los valores de retorno.

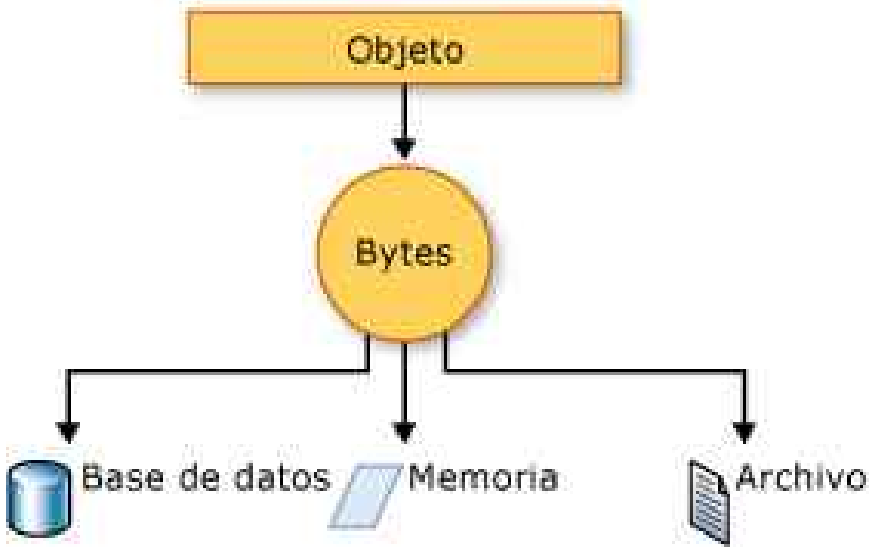


La persistencia

es una característica importante de los JavaBeans. La serialización nos permite guardar el estado de un componente en disco, abandonar el Entorno Integrado de Desarrollo (IDE) y restaurar el estado de dicho componente cuando se vuelve a correr el IDE.



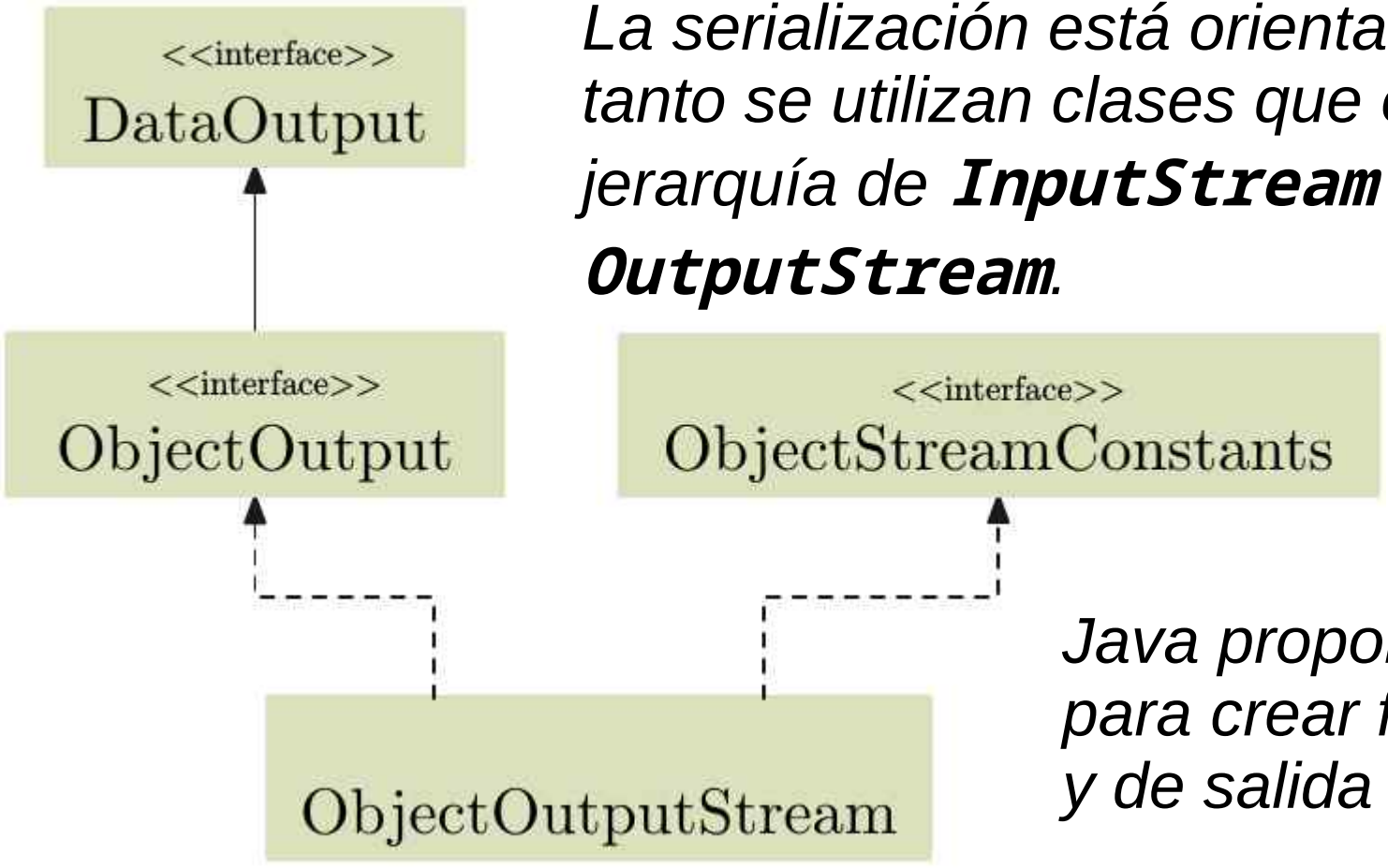
Serialización



Los objetos mantienen **referencias** a otros objetos. Estos otros objetos deben ser también almacenados y recuperados con el fin de mantener las relaciones originales. Por supuesto, todos estos objetos deben ser serializables ya que de lo contrario se lanzar´a una excepción del tipo **`NotSerializableException.`**

Para reconstruir un objeto (o conjunto de objetos) Java serializado es necesario que la clase (o clases) esté en el **classpath** con el fin de identificarla y verificarla antes de restaurar el contenido en una nueva instancia.

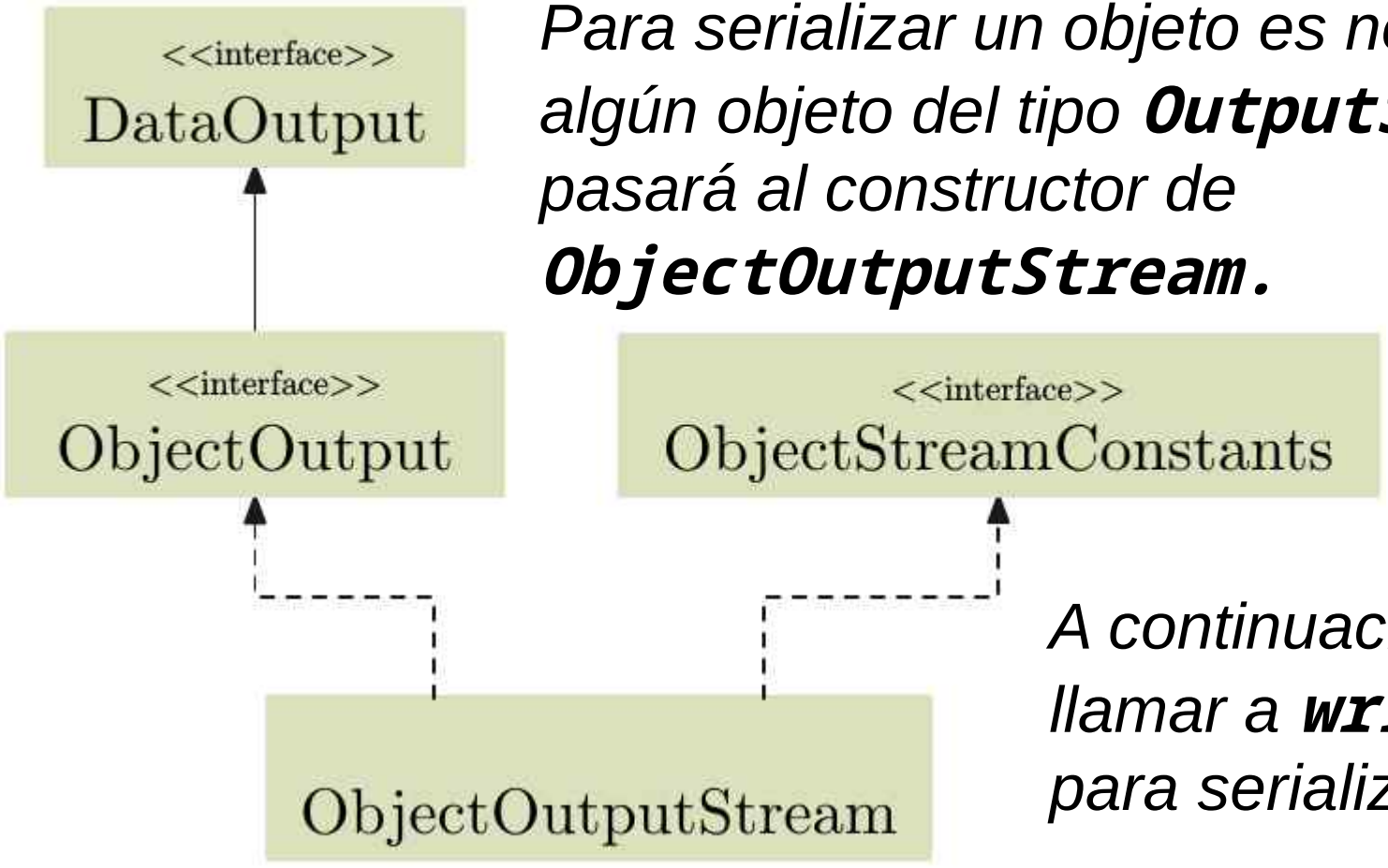
Serialización - Flujos para entrada y salida de objetos



La serialización está orientada a bytes por lo tanto se utilizan clases que estén en la jerarquía de ***InputStream*** *u* ***OutputStream****.*

Java proporciona clases para crear flujos de entrada y de salida de objetos.

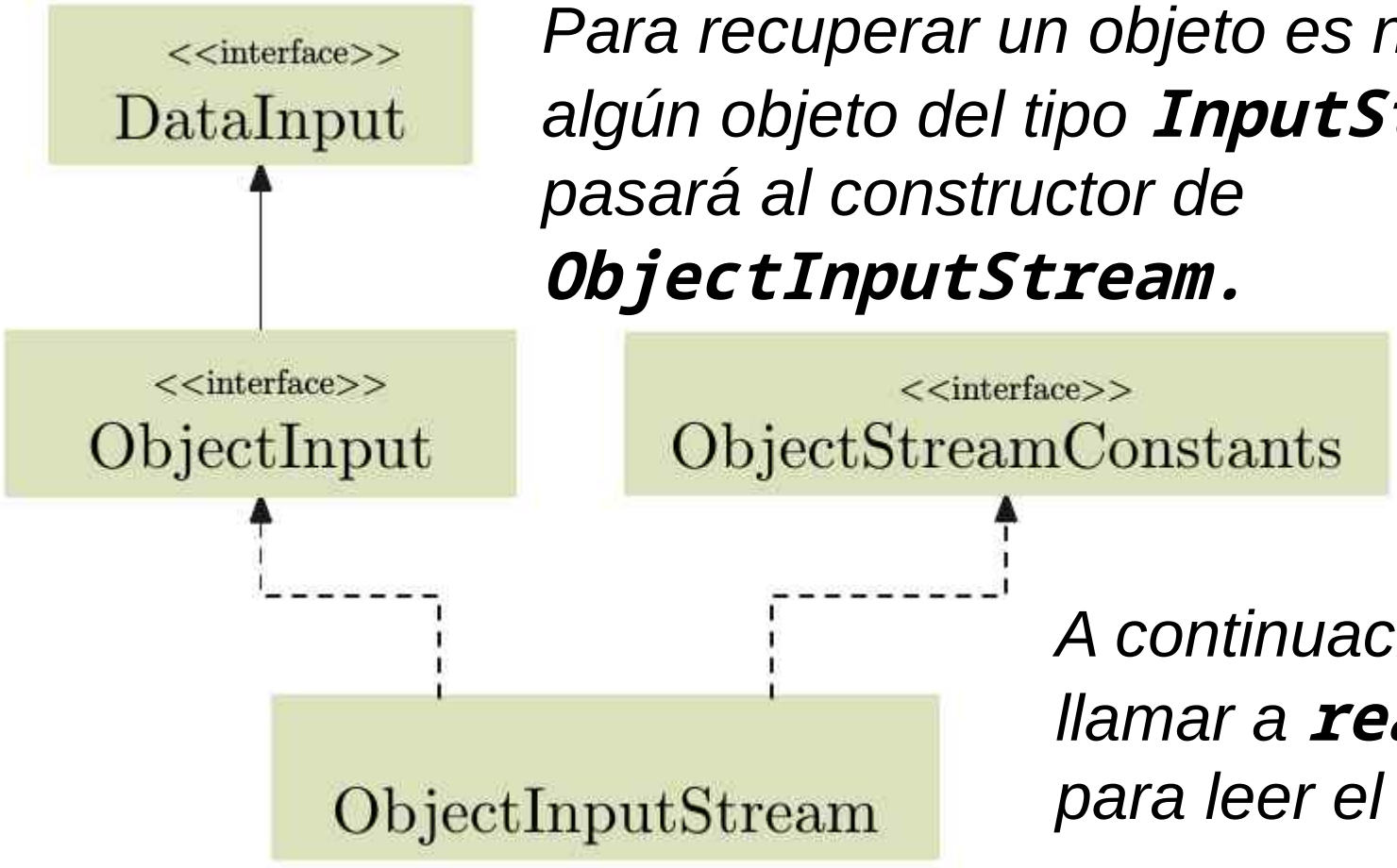
Serialización - Flujos para entrada y salida de objetos



Para serializar un objeto es necesario crear algún objeto del tipo ***OutputStream*** *que se le pasará al constructor de* ***ObjectOutputStream.***

A continuacion se puede llamar a ***writeObject()*** *para serializar el objeto.*

Serialización - Flujos para entrada y salida de objetos



Para recuperar un objeto es necesario crear algún objeto del tipo ***InputStream*** *que se le pasará al constructor de* ***ObjectInputStream.***

A continuación se puede llamar a ***readObject()*** *para leer el objeto.*

La interfaz serializable

Para que un objeto sea serializable, se debe implementar la interfaz **`java.io.Serializable`** (que lo único que hace es marcar el objeto como serializable, sin que se tenga que implementar ningún método).

- Para que un objeto sea serializable, todas sus variables de instancia han de ser serializables.
- Todos los tipos primitivos en Java son serializables por defecto (igual que los arrays y otros muchos tipos estándar).

```
package serializacion;
import java.io.Serializable;
import java.util.Date;

public class Contacto implements Serializable
{
    private String nombre;
    private String telefono;
    private String email;
    private String direccion;
    private Date nacimiento;
    private int grupo;
    private double deuda;

    public Contacto()
    {
        super();
    }
    ...
}
```

Serialización

No es necesario escribir ningún otro método. El método **`defaultWriteObject`** de la clase **`ObjectOutputStream`** realiza la serialización de los objetos de una clase.

Este método escribe en el flujo de salida todo lo necesario para reconstruir dichos objetos:

- **La clase del objeto**
- **La firma de la clase**
- **Los valores de los miembros que no tengan los modificadores static o transient, incluyendo los miembros que se refieren a otros objetos.**

El método **`defaultReadObject`** de la clase **`ObjectInputStream`** realiza la ***deserialización*** de los objetos de una clase. Este método lee el flujo de entrada y ***reconstruye los objetos*** de dicha clase.

Ejemplo

En este ejemplo la **serialización** consiste en abrir un archivo “clientes.ser” y escribir los objetos definidos como serializables, en este caso definidos en la clase “Contacto”.

```
public class ContactoOutput
{
    private FileOutputStream file;
    private ObjectOutputStream output;
    //private ByteArrayOutputStream byteArray;
    // Abrir el fichero
    public void abrir() throws IOException
    {
        file = new FileOutputStream("clientes.ser");
        output = new ObjectOutputStream(file);
    }    //output = new ObjectOutputStream(byteArray);

    // Cerrar el archivo
    public void cerrar() throws IOException
    {
        if (output != null)
            output.close();
    }

    // Escribir en el archivo
    public void escribir(Contacto contacto) throws IOException
    {
        if (output != null)
            output.writeObject(contacto);
    }
}
```

Ejemplo

Por otro lado, el proceso de **deserialización** consistirá en la lectura de los objetos serializados del archivo “contactos.dat” y mostrarlos correctamente. El archivo con los objetos serializados contactos.dat almacena los datos en un formato propio de Java, por lo que no se puede leer fácilmente con un simple editor de texto (ni editar).

```
public class ContactoInput
{
    private FileInputStream file;
    private ObjectInputStream input;

    public void abrir() throws IOException
    {
        file = new FileInputStream("contactos.dat");
        input = new ObjectInputStream(file);
    }

    public void cerrar() throws IOException
    {
        if (input != null)
            input.close();
    }

    public Contacto leer() throws IOException, ClassNotFoundException
    {
        Contacto contacto = null;
        if (input != null)
        {
            try
            {
                contacto = (Contacto) input.readObject();
            }
            catch (EOFException eof)
            { // Fin del archivo
            }
        }
        return contacto;
    }
}
```

Herencia en objetos serializables



Para serializar objetos de una jerarquía solamente la clase base tiene que implementar el **`interface Serializable.`**

Cuando se hace una deserialización, la clase que se recupera no pasa por el constructor (ver ejemplo anterior)

Los atributos declarados como transient se inicializan con su valor por defecto, dentro del contexto de inicialización de variables de una clase

Qué pasa si la superclase no ese serializable, la subclase lo será?

Los atributos heredados de las clases que no son serializables, al deserializar se iniciarán con los valores por default, no con el valor que fueron guardados.

XML



XML, siglas en inglés de eXtensible Markup Language ("lenguaje de marcas Extensible"), es un lenguaje de marcas desarrollado por el World Wide Web Consortium (W3C) utilizado para almacenar datos en forma legible. Proviene del lenguaje SGML y permite definir la gramática de lenguajes específicos (de la misma manera que HTML es a su vez un lenguaje definido por SGML) para estructurar documentos grandes.

XML se propone como un estándar para el intercambio de información estructurada entre diferentes plataformas. Se puede usar en bases de datos, editores de texto, hojas de cálculo y casi cualquier cosa imaginable.

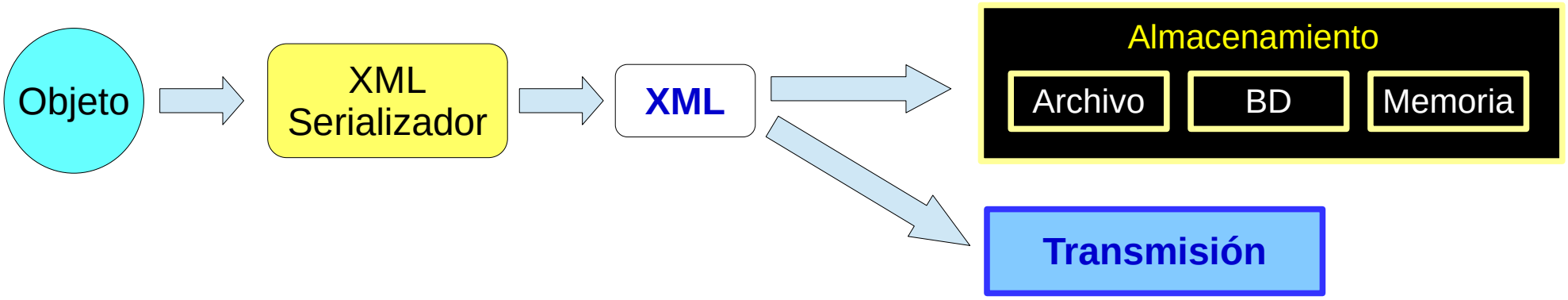
Es extensible: Después de diseñado y puesto en producción, es posible extender XML con la adición de nuevas etiquetas, de modo que se pueda continuar utilizando sin complicación alguna.

El analizador es un componente estándar, no es necesario crear un analizador específico para cada versión de lenguaje XML. Esto posibilita el empleo de cualquiera de los analizadores disponibles. De esta manera se evitan bugs y se acelera el desarrollo de aplicaciones.

Si un tercero decide usar un documento creado en XML, es sencillo entender su estructura y procesarla. Mejora la compatibilidad entre aplicaciones. Podemos **comunicar aplicaciones de distintas plataformas, sin que importe el origen de los datos**, es decir, podríamos tener una aplicación en Linux con una base de datos Postgres y comunicarla con otra aplicación en Windows y Base de Datos MS-SQL Server.

Transformamos datos en información, pues se le añade un significado concreto y los asociamos a un contexto, con lo cual tenemos flexibilidad para estructurar documentos.

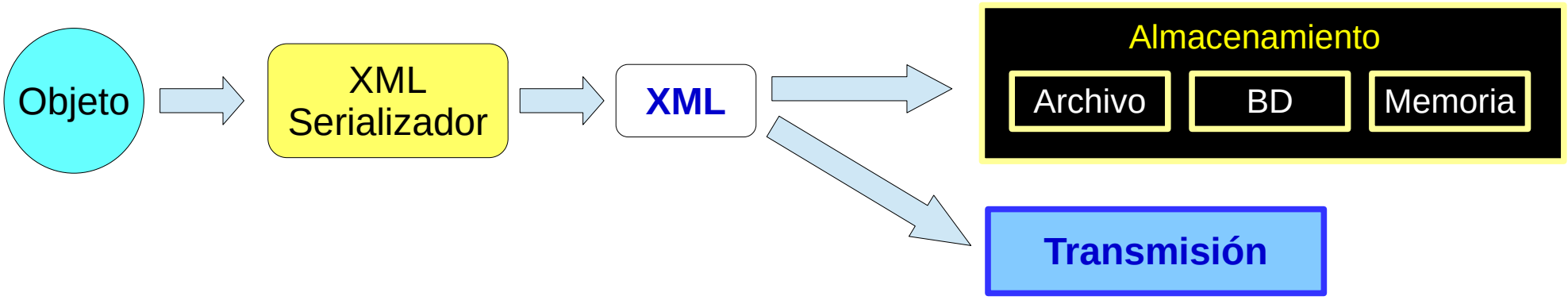
XML usado para persistir datos



La codificación en XML permite la compatibilidad de la Serialización (bajo cualquier plataforma) sin importar la versión de Java usada.

A partir del lanzamiento de Java 2 Standard Edition versión 1.4 (J2SE v1.4) los objetos Java se pudieron serializar como documentos XML. Con versiones anteriores a la versión 1.4 los objetos de Java se podían serializar solamente usando un formato binario (mediante la interfaz Serializable).

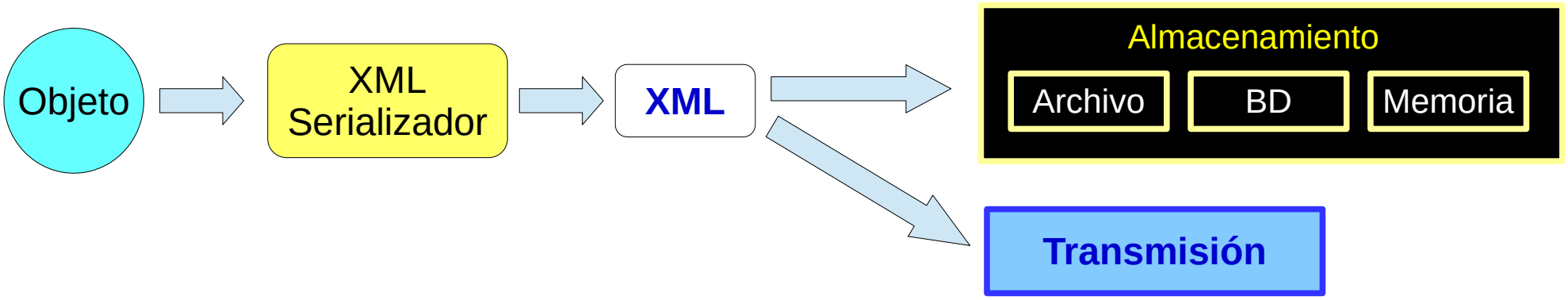
XML usado para persistir datos



Uno de los problemas de la serialización binaria es que se basa en la forma interna de la clase y la reconstrucción de objetos es frágil. No puede haber cambios entre la codificación y decodificación.

La serialización utilizando XML toma un enfoque diferente: en lugar de almacenar una representación bit a bit de los valores de campo que determinan el estado de un objeto, se almacenan los pasos necesarios para crear el objeto a través de su API pública.

XML usado para persistir datos

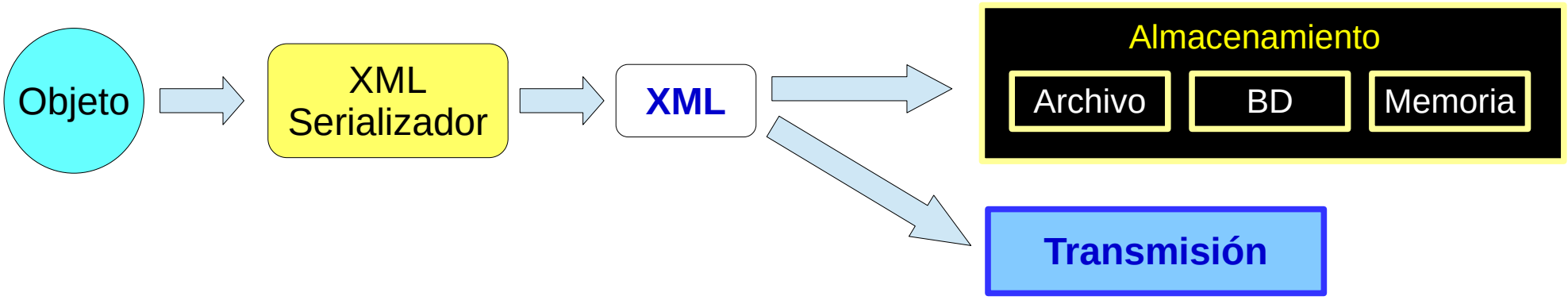


Java ofrece la clase **`java.beans.XMLEncoder`** para permitir la persistencia de objetos como documentos XML. Esta clase se encarga de convertir el objeto y todos sus datos (incluidos los campos que también son objetos) a un documento XML.

`java.beans.XMLEncoder` trabaja clonando el grafo del objeto y registrando los pasos que fueron necesarios para crear el clone.

Para reconstruir el objeto se utiliza la clase **`java.beans.XMLDecoder`**

XML usado para persistir datos



Las clases Java deben

- Ofrecer un constructor sin argumentos.
- Exponer las propiedades que constituyen el estado del objeto usando métodos get/set.
- Independencia entre las propiedades (que no haya efectos colaterales al llamar a los métodos get/set de dos propiedades distintas).

Serialización XML Ejemplo

```
public class DVD
{
    private List movies = new ArrayList();

    public DVD()
    {
    }

    public List getMovies()
    {
        return movies;
    }

    public void setMovies(List movies)
    {
        this.movies = movies;
    }

    @Override
    public String toString()
    {
        String stMovies = "";
        Iterator iter_movies = getMovies().iterator();
        while (iter_movies.hasNext())
        {
            stMovies = stMovies + ((Movie)iter_movies.next()).toString();
            stMovies = stMovies + "\n";
        }
        return stMovies;
    }
}
```

Serialización XML Ejemplo

```java
public class Movie
{
    private String name;
    private int runtime;
    private String directors;
    private int released;
    private String cast;

    public Movie()
    {
    }

    public Movie(String name, int runtime, String directors, int released, String cast)
    {
        this.name = name;
        this.runtime = runtime;
        this.directors = directors;
        this.released = released;
        this.cast = cast;
    }

    @Override
    public String toString()
    {
        // TODO Implement this method
        String datos = name + ", " + runtime + ", " +  directors  + ", " + released   + ", " +  cast;
        return datos;
    }

    // getter's y setter's
    // ...
}
```

Serialización XML Ejemplo

```
public class SerializeToXML
{
    private static final String SERIALIZED_FILE_NAME = "dvd.xml";

    public static void main(String args[])
    {
        Movie bourneIndentity = new Movie("The Bourne Identity", 119, "Doug Liman", 2002, "Matt Damon, Franka Potente");
        Movie bourneSupermacy = new Movie("The Bourne Supremacy", 108, "Paul Greengrass", 2004, "Matt Damon, Franka Potente, Joan Allen");
        Movie bourneUltimatum = new Movie("The Bourne Ultimatum", 115, "Paul Greengrass", 2007, "Matt Damon, Edgar Ramirez, Joan Allen");
        Movie bourneLegacy = new Movie("The Bourne Legacy", 135, "Tony Gilroy", 2012, "Jeremy Renner, Rachel Weisz, Edward Norton");
        List moviesList = new ArrayList();
        moviesList.add(bourneIndentity);
        moviesList.add(bourneSupermacy);
        moviesList.add(bourneUltimatum);
        moviesList.add(bourneLegacy);
        DVD bourneSeries = new DVD();
        bourneSeries.setMovies(moviesList);
        XMLEncoder encoder = null;
        try
        {
            encoder = new XMLEncoder(new BufferedOutputStream(new FileOutputStream(SERIALIZED_FILE_NAME)));
        }
        catch (FileNotFoundException fileNotFound)
        {
            System.out.println("ERROR: While Creating or Opening the File dvd.xml");
        }
        encoder.writeObject(bourneSeries);
        encoder.close();
    }
}
```

Serialización XML Ejemplo

```java
public class DeserializeFromXML
{
    private static final String SERIALIZED_FILE_NAME = "dvd.xml";

    public static void main(String[] args)
    {
        XMLDecoder decoder = null;
        try
        {
            decoder = new XMLDecoder(new BufferedInputStream(new FileInputStream(SERIALIZED_FILE_NAME)));
        }
        catch (FileNotFoundException e)
        {
            System.out.println("ERROR: File dvd.xml not found");
        }
        DVD bourneSeries = (DVD) decoder.readObject();
        System.out.println(bourneSeries);
    }

}
```

bourneSeries.toString()

```
The Bourne Identity, 119, Doug Liman, 2002, Matt Damon, Franka Potente
The Bourne Supremacy, 108, Paul Greengrass, 2004, Matt Damon, Franka Potente, Joan Allen
The Bourne Ultimatum, 115, Paul Greengrass, 2007, Matt Damon, Edgar Ramirez, Joan Allen
The Bourne Legacy, 135, Tony Gilroy, 2012, Jeremy Renner, Rachel Weisz, Edward Norton

Process exited with exit code 0.
```

Archivo dvd.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<java version="1.7.0_79" class="java.beans.XMLDecoder">
 <object class="project1.DVD" id="DVD0">
  <void property="movies">
   <void method="add">
    <object class="project1.Movie">
     <void property="cast">
      <string>Matt Damon, Franka Potente</string>
     </void>
     <void property="directors">
      <string>Doug Liman</string>
     </void>
     <void property="name">
      <string>The Bourne Identity</string>
     </void>
     <void property="released">
      <int>2002</int>
     </void>
     <void property="runtime">
      <int>119</int>
     </void>
    </object>
   </void>
```

```
<void method="add">
 <object class="project1.Movie">
  <void property="cast">
   <string>Matt Damon, Franka Potente, Joan Allen</string>
  </void>
  <void property="directors">
   <string>Paul Greengrass</string>
  </void>
  <void property="name">
   <string>The Bourne Supremacy</string>
  </void>
  <void property="released">
   <int>2004</int>
  </void>
  <void property="runtime">
   <int>108</int>
  </void>
 </object>
</void>
```

```
<void method="add">
 <object class="project1.Movie">
  <void property="cast">
   <string>Matt Damon, Edgar Ramirez, Joan Allen</string>
  </void>
  <void property="directors">
   <string>Paul Greengrass</string>
  </void>
  <void property="name">
   <string>The Bourne Ultimatum</string>
  </void>
  <void property="released">
   <int>2007</int>
  </void>
  <void property="runtime">
   <int>115</int>
  </void>
 </object>
</void>
```

```
    <void method="add">
     <object class="project1.Movie">
      <void property="cast">
       <string>Jeremy Renner, Rachel Weisz, Edward Norton</string>
      </void>
      <void property="directors">
       <string>Tony Gilroy</string>
      </void>
      <void property="name">
       <string>The Bourne Legacy</string>
      </void>
      <void property="released">
       <int>2012</int>
      </void>
      <void property="runtime">
       <int>135</int>
      </void>
     </object>
    </void>
   </void>
  </object>
 </java>
```